

PyBrainViewer User Manual

AI-Powered Multi-Function Brain Visual Tool

Version 0.2.0

Tang Zhihao
ShanghaiTech University

2025

Contents

1	Introduction	3
1.1	Background	3
1.2	Features	3
2	Installation	3
2.1	System Requirements	3
2.2	Install from Source	4
2.3	Install from PyPI	4
2.4	Optional Dependencies	4
3	Quick Start	4
3.1	CLI Mode	4
3.2	GUI Mode	4
3.3	TUI Mode	5
3.4	Python API	5
4	File Formats	5
4.1	Surface Format (.nv)	5
4.2	Node Format (.node)	5
4.3	Edge Format (.edge)	5
4.4	Volume Formats	5
5	CLI Reference	6
5.1	Visualize Commands	6
5.1.1	Surface Visualization	6
5.1.2	Node Visualization	6
5.2	Process Commands	6
5.2.1	Connectivity Analysis	6
5.2.2	Image Registration	6
5.3	AI Commands	7
6	GUI Reference	7
6.1	Main Window	7
6.2	Menu Bar	7
6.2.1	File Menu	7
6.2.2	View Menu	7
6.2.3	AI Menu	8
6.2.4	Process Menu	8
7	Python API Reference	8
7.1	Core Module	8
7.1.1	Surface Parsing	8
7.1.2	Node Parsing	8
7.1.3	Edge Parsing	8
7.1.4	Volume-to-Surface Mapping	9
7.2	Visualization Module	9
7.3	Processing Module	9
7.4	AI Module	9
8	BNV Compatibility	10

9	Troubleshooting	10
9.1	Common Issues	10
9.2	Getting Help	10
10	Appendix	11
10.1	Keyboard Shortcuts	11
10.2	Colormap Reference	11

1 Introduction

PyBrainViewer is a modern, cross-platform Python tool for brain imaging data analysis and 3D visualization. It serves as a comprehensive replacement for the MATLAB-based BrainNet Viewer, adding modern features while maintaining full compatibility with existing file formats.

1.1 Background

Functional Magnetic Resonance Imaging (fMRI) is a powerful tool for studying brain activity. The Blood Oxygen Level Dependent (BOLD) effect allows researchers to measure neural activity by detecting changes in blood flow. However, raw fMRI data requires significant processing and visualization to be useful for research.

Existing tools like BrainNet Viewer (MATLAB-based, last updated 2019) have served the neuroscience community well but suffer from several limitations:

- Requires expensive MATLAB license
- Outdated user interface
- Single-function focus (visualization only)
- Limited to Windows platform
- Poor code quality (5000+ lines in single file)

PyBrainViewer addresses these limitations with a modern Python implementation.

1.2 Features

- **Three Interfaces:** CLI (Click), TUI (Textual), GUI (PyQt6)
- **3D Brain Visualization:** Surface, node, and edge rendering
- **Volume-to-Surface Mapping:** 7 algorithms
- **Image Processing:** DICOM/NIfTI conversion, normalization, quality metrics
- **Connectivity Analysis:** fMRI correlation matrices
- **AI Features:** Super-resolution, brain segmentation, LLM assistant
- **External Tool Integration:** FreeSurfer, SPM, Elastix
- **BNV Compatibility:** Full .nv, .node, .edge format support
- **Cross-Platform:** Windows, macOS, Linux

2 Installation

2.1 System Requirements

- Python 3.10 or higher
- 4GB RAM (8GB recommended for large datasets)
- OpenGL 3.0+ compatible graphics card
- For AI features: NVIDIA GPU with CUDA (optional)

2.2 Install from Source

Listing 1: Installation from source

```
1 git clone https://github.com/tzhazuma/PyBNT.git
2 cd PyBNT
3 pip install -e .
```

2.3 Install from PyPI

Listing 2: Installation via pip

```
1 pip install pybnt
```

2.4 Optional Dependencies

Listing 3: Installing optional dependencies

```
1 # AI features (PyTorch, TensorFlow)
2 pip install pybnt[ai]
3
4 # External tool integration
5 pip install pybnt[external]
6
7 # Development tools
8 pip install pybnt[dev]
```

3 Quick Start

3.1 CLI Mode

The quickest way to get started is the CLI:

Listing 4: Basic CLI usage

```
1 # View help
2 pybnt --help
3
4 # Visualize a brain surface
5 pybnt visualize surface pybnt/data/templates/BrainMesh_ICBM152.nv
6
7 # Compute connectivity matrix
8 pybnt process compute-connectivity fmri_4d.nii --output matrix.edge
```

3.2 GUI Mode

Launch the graphical interface:

Listing 5: Launching GUI

```
1 pybnt-gui
```

Or alternatively:

```
1 python -m pybnt
```

3.3 TUI Mode

Launch the terminal interface:

Listing 6: Launching TUI

```
1 pybnt tui
```

3.4 Python API

Listing 7: Python API usage

```
1 import pybnt
2
3 # Parse a surface file
4 nodes, tris = pybnt.core.surface.parse_surface(
5     "pybnt/data/templates/BrainMesh_ICBM152.nv"
6 )
7 print(f"Loaded {len(nodes)} vertices and {len(tris)} triangles")
```

4 File Formats

4.1 Surface Format (.nv)

The .nv format is BrainNet Viewer's native mesh format, consisting of:

- Line 1: Number of vertices
- Vertex lines: x y z coordinates
- Next line: Number of triangles
- Triangle lines: i_1 i_2 i_3 (1-indexed indices)

4.2 Node Format (.node)

Six-column tab-delimited format:

- Columns 1-3: x , y , z coordinates (MNI space)
- Column 4: Color index or continuous value
- Column 5: Node size
- Column 6: Node label (or '-' for no label)

4.3 Edge Format (.edge)

NCEN adjacency or correlation matrix in space/tab-delimited ASCII format. Supports binary, weighted, directed, or undirected matrices.

4.4 Volume Formats

NIfTI (.nii), compressed NIfTI (.nii.gz), DICOM (.dcm), and text-based volumes are supported for volume-to-surface mapping.

5 CLI Reference

5.1 Visualize Commands

5.1.1 Surface Visualization

Listing 8: pybnt visualize surface

```

1 pybnt visualize surface [OPTIONS] SURFACE_PATH
2
3 Options:
4   --smooth           Smooth the surface mesh
5   --light TEXT       Lighting preset (default/head/tail/left/right/none)
6   --colormap TEXT     Surface colormap (default: white)
7   --opacity FLOAT     Surface opacity (0-1)
8   --output, -o PATH   Save screenshot to file

```

5.1.2 Node Visualization

Listing 9: pybnt visualize nodes

```

1 pybnt visualize nodes [OPTIONS] NODE_PATH
2
3 Options:
4   -e, --edge-path PATH      Edge file (.edge)
5   -s, --surface-path PATH   Surface file (.nv)
6   --node-color TEXT         Node color (default: red)
7   --edge-color TEXT         Edge color (default: blue)
8   --surface-color TEXT      Surface color (default: white)
9   --node-scale FLOAT        Node scaling factor
10  --output, -o PATH          Save screenshot

```

5.2 Process Commands

5.2.1 Connectivity Analysis

Listing 10: pybnt process compute-connectivity

```

1 pybnt process compute-connectivity [OPTIONS] IMAGE_PATH
2
3 Options:
4   --output, -o PATH   Output path for connectivity matrix
5   --method TEXT       Connectivity method (default: correlation)

```

5.2.2 Image Registration

Listing 11: pybnt process register

```

1 pybnt process register [OPTIONS] FIXED_IMAGE MOVING_IMAGE
2
3 Options:
4   -o, --output-dir PATH   Output directory (default: ./
    ↪ registration_output)

```

5.3 AI Commands

Listing 12: pybnt ai commands

```

1 # Brain segmentation
2 pybnt ai segment [OPTIONS] IMAGE_PATH
3   -o, --output PATH    Output directory
4
5 # Image correction
6 pybnt ai correct [OPTIONS] IMAGE_PATH
7   -o, --output PATH    Output directory
8
9 # AI assistant
10 pybnt ai ask [OPTIONS]
11   -m, --message TEXT    Question to ask (required)
12   -k, --api-key TEXT     API key
13   --model TEXT          LLM model name (default: qwen-plus)

```

6 GUI Reference

6.1 Main Window

The GUI consists of:

- **Menu Bar:** File, Process, View, AI, Help
- **Toolbar:** Quick access to common actions
- **Input Panel:** File path fields with browse buttons
- **Processing Panel:** Checkboxes for processing options
- **Log Panel:** Processing status and messages
- **Status Bar:** Current application state

6.2 Menu Bar

6.2.1 File Menu

- Open Surface (.nv) – Ctrl+O
- Open Node (.node) – Ctrl+N
- Open Edge (.edge) – Ctrl+E
- Open Image (NIfTI/DICOM) – Ctrl+I
- Save Screenshot – Ctrl+S
- Exit – Ctrl+Q

6.2.2 View Menu

- Layout options (Lateral, Medial, Ventral, etc.)
- Toggle Dark Mode – Ctrl+D

6.2.3 AI Menu

- Brain Segmentation
- Denoise Image
- Ask AI Assistant – Ctrl+A

6.2.4 Process Menu

- Compute Connectivity
- Register Images
- Super-Resolution

7 Python API Reference

7.1 Core Module

7.1.1 Surface Parsing

Listing 13: pybnt.core.surface

```
1 from pybnt.core.surface import parse_surface
2
3 nodes, tris = parse_surface("BrainMesh_ICBM152.nv")
4 # nodes: list of [x, y, z] coordinates
5 # tris: list of [i1, i2, i3] triangle indices
```

7.1.2 Node Parsing

Listing 14: pybnt.core.nodes

```
1 from pybnt.core.nodes import parse_node_file
2
3 nodes = parse_node_file("Node_AAL90.node")
4 # Each node: {"x": float, "y": float, "z": float,
5 #             "value": float, "size": float, "label": str}
```

7.1.3 Edge Parsing

Listing 15: pybnt.core.edges

```
1 from pybnt.core.edges import parse_edge_file
2
3 matrix = parse_edge_file("Edge_AAL90.edge")
4 # Returns numpy array of shape (N, N)
```

7.1.4 Volume-to-Surface Mapping

Listing 16: pybnt.core.volume

```

1 from pybnt.core.volume import map_volume_to_surface
2
3 vertex_data = map_volume_to_surface(
4     "stat_map.nii",
5     "BrainMesh_ICBM152.nv",
6     algorithm="gaussian",
7     radius=5.0
8 )
9 # Returns numpy array of shape (num_vertices,)
```

7.2 Visualization Module

Listing 17: pybnt.visualization

```

1 from pybnt.visualization.plotter import draw_surface, draw_nodes
2 from pybnt.visualization.colormaps import BUILTIN_COLORMAPS
3 from pybnt.visualization.layouts import CAMERA_PRESETS
4
5 # Draw surface
6 plotter = draw_surface("BrainMesh_ICBM152.nv", show=True)
7
8 # Draw nodes with edges and surface
9 plotter = draw_nodes(
10     "Node_AAL90.node",
11     edge_path="Edge_AAL90.edge",
12     surface_path="BrainMesh_ICBM152.nv"
13 )
```

7.3 Processing Module

Listing 18: pybnt.processing

```

1 from pybnt.processing.connectivity import compute_correlation_matrix
2 from pybnt.processing.registration import register_image
3
4 # Compute connectivity
5 matrix = compute_correlation_matrix("fMRI_4D.nii")
6
7 # Register images
8 result = register_image("fixed.nii", "moving.nii", "./output")
```

7.4 AI Module

Listing 19: pybnt.ai

```

1 from pybnt.ai.llm import ask_llm
2 from pybnt.ai.segmentation import segment_brain
3
4 # Ask brain science question
5 response = ask_llm([
6     {"role": "user", "content": "What is the default mode network?"}
7 ], api_key="your-api-key")
```

```

8
9 # Segment brain
10 result = segment_brain("brain.nii", "./segmentation_output")

```

8 BNV Compatibility

PyBrainViewer is designed as a drop-in replacement for BrainNet Viewer (BNV). All BNV file formats are supported:

Format	Extension	BNV Compatible
Surface	.nv	Full
FreeSurfer surface	.pial	Full
GIFTI surface	.gii	Full
Wavefront OBJ	.obj	Full
Node file	.node	Full
Edge file	.edge	Full
NIfTI volume	.nii	Full

Table 1: File format compatibility with BrainNet Viewer

9 Troubleshooting

9.1 Common Issues

1. **Missing Dependencies:** Run `pip install -r requirements.txt`
2. **OpenGL Errors:** Ensure your graphics driver supports OpenGL 3.0+
3. **FreeSurfer Not Found:** Install FreeSurfer or use `FREESURFER_HOME` environment variable
4. **Qt Platform Plugin Error:** Install `libxcb-cursor0` on Linux
5. **AI Model Not Loading:** Ensure model weights are downloaded

9.2 Getting Help

- GitHub Issues: <https://github.com/tzhazuma/PyBNT/issues>
- Documentation: <https://github.com/tzhazuma/PyBNT>

10 Appendix

10.1 Keyboard Shortcuts

Shortcut	Action
Ctrl+O	Open surface file
Ctrl+N	Open node file
Ctrl+E	Open edge file
Ctrl+I	Open image file
Ctrl+S	Save screenshot
Ctrl+D	Toggle dark mode
Ctrl+Q	Exit
Ctrl+A	Ask AI assistant

Table 2: GUI keyboard shortcuts

10.2 Colormap Reference

Name	Description
jet	Rainbow colormap (blue to red)
hot	Black to red to yellow to white
cold	Cyan to blue to dark blue
spectral	Full spectral range
bone	Grayscale with blue tint
copper	Black to copper
pink	Grayscale with pink tint
gray	Grayscale only